CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology and Engineering

Devang Patel Institute of Advance Technology and Research

Department of Information Technology

BIG DATA ANALYTICS (CSUE301)

PRACTICAL 4

Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce

1. OBJECTIVE

The objective of this practical is to implement a Hadoop MapReduce program for analyzing NCDC weather records. The program reads yearly weather observations, extracts the year and temperature values, groups records by year and finds the maximum recorded temperature for each year. The result is stored and verified in HDFS.

2. AIM

To upload NCDC weather data to HDFS and implement a Java MapReduce program that finds the maximum temperature recorded for each year.

3. PROBLEM DEFINITION

A meteorological research organization maintains a large collection of historical weather records from different stations. Processing such records one by one on a single machine becomes difficult as the dataset grows. In this practical, Hadoop is used to store the weather data in HDFS and MapReduce is used to process the records in parallel. The Mapper extracts the year and temperature, while the Reducer finds the maximum temperature for each year.

5. TOOLS / TECHNOLOGY USED

- Ubuntu Linux / WSL Terminal
- OpenJDK 11.0.31
- Apache Hadoop 3.4.1
- HDFS
- YARN
- Java MapReduce
- Windows folder mounted through WSL under /mnt/c/

6. WORKING FOLDER

The practical was performed in a separate Practical 4 folder so that the earlier Practical 3 Word Count files remained unchanged.

```
cd "/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature"
pwd
ls -lah
```

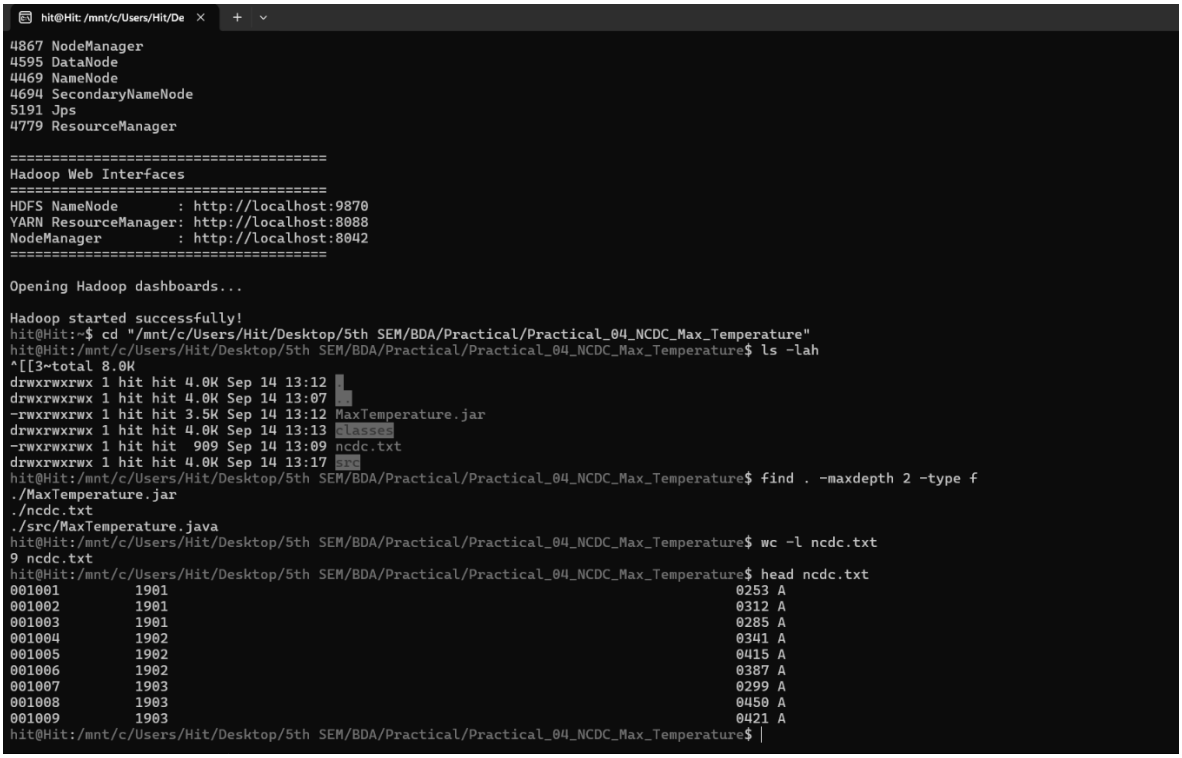


Figure 1: Hadoop environment, Practical 4 working folder and test NCDC dataset.

7. INPUT DATA USED

A small fixed-width NCDC test file named ncdc.txt was used first to validate the program. The test file contains 9 weather records for the years 1901, 1902 and 1903. The temperature values are represented in tenths of a degree Celsius, so a stored value such as 312 represents 31.2 °C.

The test records contain the main fields required by the program: station identifier, year, temperature and quality code.

head ncdc.txt

wc -l ncdc.txt

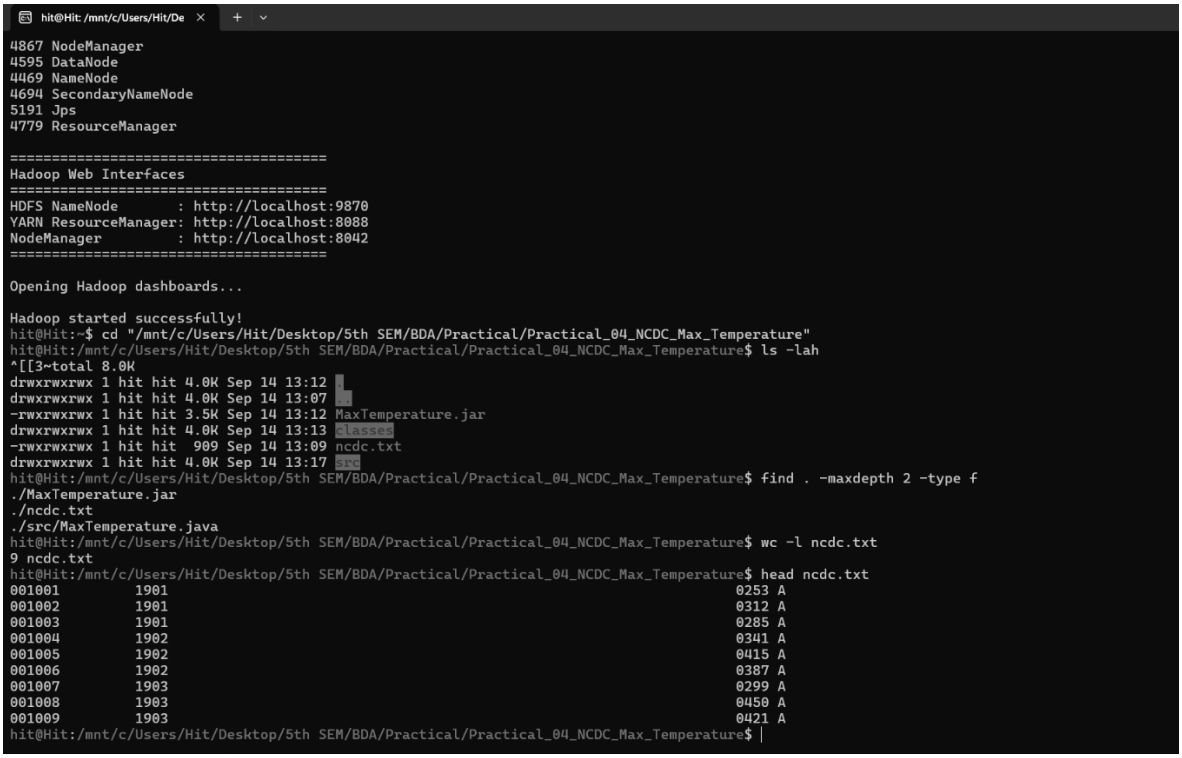


Figure 2: Verification of the local NCDC test file and its 9 records.

8. NCDC RECORD STRUCTURE USED

Field	Position used by program	Purpose
Year	15-18	Used as the MapReduce key.
Temperature	87-91	Used as the numeric value for maximum calculation.
Quality	92	Used to ignore unsuitable or missing observations.

The Mapper reads the record using fixed-width positions rather than comma-separated fields. Missing temperature value 9999 is ignored. The current program accepts the NCDC quality codes 0, 1, 4, 5 and 9 for the test run.

9. STARTING HADOOP

Before working with HDFS, the Hadoop services were started and checked with jps. The running NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager confirm that the single-node Hadoop environment is available.

start-hadoop

jps

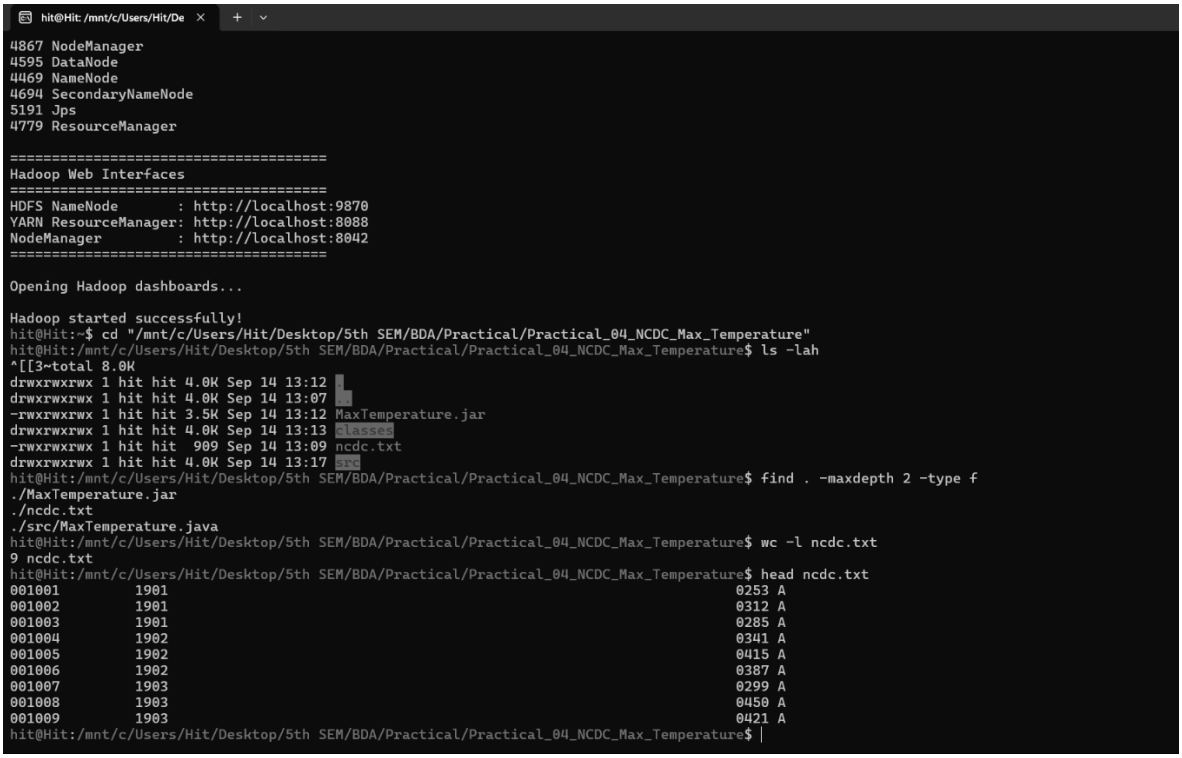


Figure 3: Hadoop services running in the WSL environment.

10. UPLOADING THE DATASET TO HDFS

An input directory was created in HDFS and the NCDC file was uploaded. The file was then listed and read directly from HDFS to verify the upload.

```
hdfs dfs -mkdir -p /user/hit/ncdc_p4/input

hdfs dfs -put ncdc.txt /user/hit/ncdc_p4/input/

hdfs dfs -ls /user/hit/ncdc_p4/input

hdfs dfs -cat /user/hit/ncdc_p4/input/ncdc.txt | head
```

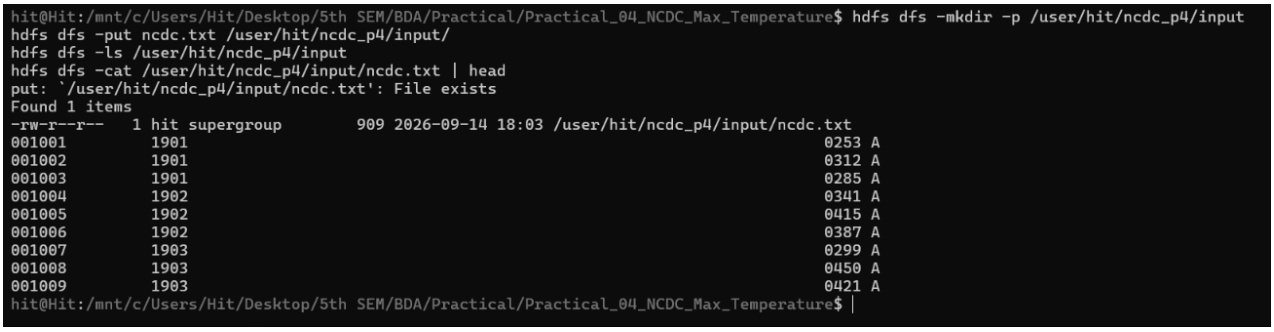


Figure 4: Uploading the NCDC dataset to HDFS and verifying the stored file.

11. MAPPER IMPLEMENTATION

The Mapper processes one weather record at a time. It extracts the year from the fixed-width record and reads the temperature value. Invalid, missing or unsuitable observations are skipped. For every valid observation, it generates the intermediate key-value pair (year, temperature).

11.1 Mapper Logic

- Read one NCDC weather record.
- Check that the record is long enough for the required fields.
- Extract the year using substring(15, 19).
- Extract the temperature using substring(87, 92).
- Extract the quality code from position 92.
- Ignore missing temperature value 9999.
- Ignore records whose quality code is not accepted.
- Write the intermediate pair (year, temperature).

Example: three records for 1901 may produce (1901, 253), (1901, 312) and (1901, 285). Hadoop automatically groups the same year during Shuffle and Sort.

12. REDUCER IMPLEMENTATION

The Reducer receives one year together with all temperature values belonging to that year. It compares the values one by one and keeps the largest value. After all values are checked, it writes the year and its maximum temperature.

12.1 Reducer Logic

- Receive the year as the key and all temperatures as values.
- Start the maximum with Integer.MIN_VALUE.
- Compare each temperature with the current maximum.
- Replace the maximum whenever a larger value is found.
- Write the final pair (year, maximum temperature).

For example, if the Reducer receives 1901 -> [253, 312, 285], it produces 1901 -> 312.

13. COMPLETE JAVA PROGRAM

```
import java.io.IOException;


import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;


public class MaxTemperature {


  private static final int MISSING = 9999;
```

```
public static class TemperatureMapper
        extends Mapper<Object, Text, Text, IntWritable> {

    private final Text year = new Text();
    private final IntWritable temperature = new IntWritable();

    @Override
    public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

        String line = value.toString();

        if (line.length() < 93) {
            return;
        }

        try {
            String yearValue = line.substring(15, 19);
            String temperatureValue = line.substring(87, 92);
            String quality = line.substring(92, 93);

            int airTemperature = Integer.parseInt(temperatureValue);

            if (airTemperature == MISSING) {
                return;
            }

            if (!quality.matches("[01459]")) {
                return;
            }

            year.set(yearValue);
            temperature.set(airTemperature);

            context.write(year, temperature);

        } catch (NumberFormatException e) {
            // Ignore malformed records
        }
```

```
    }
  }

  public static class MaxTemperatureReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {

    private final IntWritable result = new IntWritable();

    @Override
    public void reduce(Text key,
                       Iterable<IntWritable> values,
                       Context context)
        throws IOException, InterruptedException {

      int maxTemperature = Integer.MIN_VALUE;

      for (IntWritable value : values) {
        int currentTemperature = value.get();

        if (currentTemperature > maxTemperature) {
          maxTemperature = currentTemperature;
        }
      }

      result.set(maxTemperature);
      context.write(key, result);
    }
  }

  public static void main(String[] args) throws Exception {

    if (args.length != 2) {
      System.err.println(
        "Usage: MaxTemperature <input path> <output path>"
      );
      System.exit(2);
    }

    Configuration conf = new Configuration();
```

```
        Job job = Job.getInstance(
                conf,
                "Maximum Temperature"
        );

        job.setJarByClass(MaxTemperature.class);

        job.setMapperClass(TemperatureMapper.class);
        job.setReducerClass(MaxTemperatureReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(
                job,
                new Path(args[0])
        );

        FileOutputFormat.setOutputPath(
                job,
                new Path(args[1])
        );

        System.exit(
                job.waitForCompletion(true) ? 0 : 1
        );
    }
}
```

14. COMPILATION AND CLASS FILE VERIFICATION

The Java source file was compiled using the Hadoop classpath and the generated class files were placed inside the classes folder.

```
rm -rf classes/*
javac -classpath "$(hadoop classpath)" -d classes src/MaxTemperature.java
find classes -type f
```

```
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ python3 - <<'PY'
text = open("src/MaxTemperature.java").read()
print("Opening braces :", text.count("{"))
print("Closing braces :", text.count("}"))
PY
Opening braces : 14
Closing braces : 13
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ echo '}' >> src/MaxTemperature.java
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ python3 - <<'PY'
text = open("src/MaxTemperature.java").read()
print("Opening braces :", text.count("{"))
print("Closing braces :", text.count("}"))
PY
Opening braces : 14
Closing braces : 14
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ rm -rf classes/*
javac -classpath "$(hadoop classpath)" -d classes src/MaxTemperature.java
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ find classes -type f
classes/MaxTemperature$MaxTemperatureReducer.class
classes/MaxTemperature$TemperatureMapper.class
classes/MaxTemperature.class
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$
```

Figure 5: Successful compilation of MaxTemperature.java and generation of the Mapper, Reducer and driver class files.

15. JAR CREATION

After successful compilation, the generated class files were packaged into MaxTemperature.jar. The JAR contents were also checked before running the Hadoop job.

```
jar -cvf MaxTemperature.jar -C classes .

ls -lh MaxTemperature.jar

jar tf MaxTemperature.jar
```

```
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ ls -lh MaxTemperature.jar
-rwxrwxrwx 1 hit hit 3.5K Sep 14 18:02 MaxTemperature.jar
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$ jar tf MaxTemperature.jar
META-INF/
META-INF/MANIFEST.MF
MaxTemperature$MaxTemperatureReducer.class
MaxTemperature$TemperatureMapper.class
MaxTemperature.class
hit@Hit:/mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_04_NCDC_Max_Temperature$
```

Figure 6: Creation and verification of MaxTemperature.jar.

16. MAPREDUCE JOB EXECUTION

The Hadoop MapReduce job was executed using the HDFS input directory and a separate HDFS output directory. The old output directory was removed before the run so that Hadoop could create a fresh result.

```
hdfs dfs -rm -r -f /user/hit/ncdc_p4/output


hadoop jar MaxTemperature.jar MaxTemperature \
/user/hit/ncdc_p4/input \
/user/hit/ncdc_p4/output
```

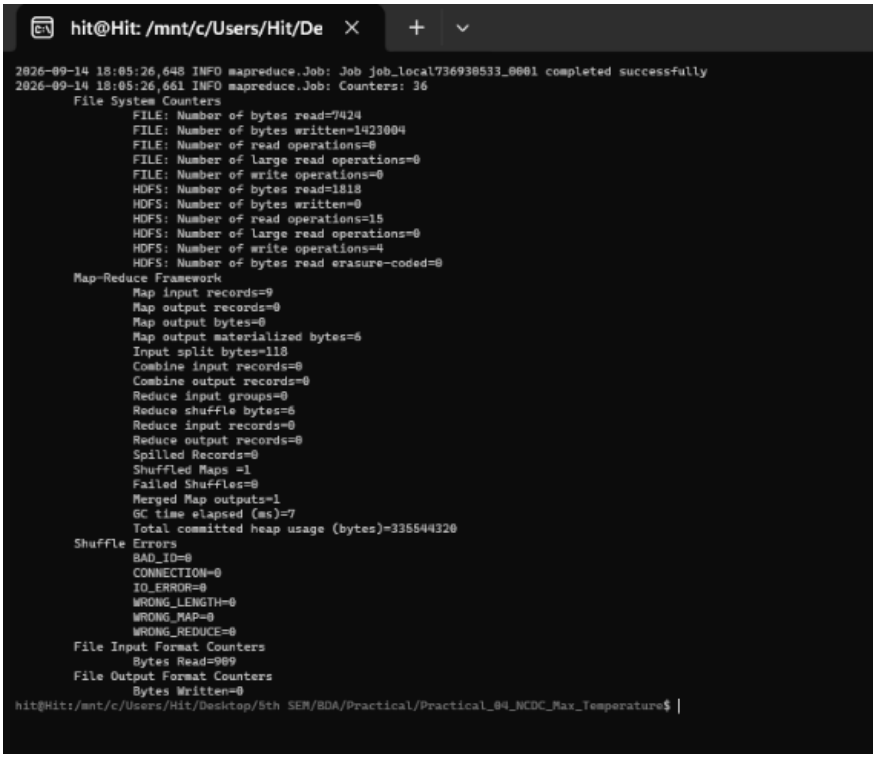


Figure 7: Captured MapReduce execution showing successful job completion and Hadoop counters.

17. EXECUTION OBSERVATION

The captured Hadoop job completed successfully at the framework level. However, the counters show Map output records = 0 and Reduce input groups = 0. This happened because the captured test input used quality code 'A', while the Mapper was configured to accept only the quality codes 0, 1, 4, 5 and 9. Therefore, the Mapper correctly filtered all nine test records and produced an empty result.

This is an input-validation issue, not a compilation or Hadoop startup issue. The Java program compiled successfully, the JAR was created correctly, and the MapReduce job itself finished without a framework failure.

18. OUTPUT VERIFICATION

The output directory was created by Hadoop and contains the normal MapReduce result file. In the captured run, part-r-00000 is empty because the Mapper emitted no records.

```
hdfs dfs -ls /user/hit/ncdc_p4/output

hdfs dfs -cat /user/hit/ncdc_p4/output/part-r-00000
```

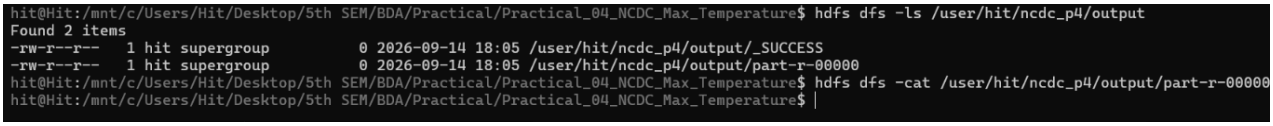


Figure 8: HDFS output directory after the captured job. The result file is present but empty because all test records were filtered.

19. CORRECTED TEST RUN - EXPECTED RESULT

For validation, the same nine test records can be rerun after replacing the quality code A with an accepted code such as 0. With the current temperature values, the expected yearly maximums are:

Year	Temperatures in test data	Maximum stored value	Maximum in °C
1901	253, 312, 285	312	31.2 °C
1902	341, 415, 387	415	41.5 °C
1903	299, 450, 421	450	45.0 °C

The hottest year in the corrected test run would therefore be 1903 with a maximum temperature of 45.0 °C. These are validation values for the small test file, not a claim about the full NCDC dataset.

20. FINDING THE HOTTEST YEAR OVERALL

Once the yearly maximum output is available, the largest yearly value can be found by sorting the reducer output in descending order of the second column.

```
hdfs dfs -cat /user/hit/ncdc_p4/output/part-r-00000 \
| sort -k2,2nr | head -1
```

For the corrected nine-record test file, this would return 1903 with a stored value of 450, equivalent to 45.0 °C.

21. KEY QUESTIONS / ANALYSIS

Q1. How is weather data parsed and processed using MapReduce?

The NCDC record is a fixed-width text record. The Mapper reads each line, extracts the year and temperature using their fixed character positions, checks the quality field and emits (year, temperature). Hadoop groups the same year during Shuffle and Sort. The Reducer then receives all temperatures for that year and chooses the maximum.

Q2. What key-value pairs are generated by the Mapper?

For every valid observation, the Mapper generates a pair such as (1901, 253) or (1901, 312). The year is the key and the recorded temperature is the value.

Q3. How does the Reducer identify the maximum temperature for a given year?

The Reducer starts with a very small initial value and compares every temperature in the group with the current maximum. Whenever a larger value is found, the maximum is updated. After all values are processed, the final maximum is written.

Q4. What challenges arise when processing large climate datasets?

Large weather datasets may contain many files, different stations, missing values, invalid observations and large volumes of records. Fixed-width parsing, data quality checks, storage management and shuffle traffic also need to be handled carefully.

Q5. How does Hadoop improve scalability and performance for weather data analytics?

HDFS distributes large files across DataNodes and MapReduce allows different parts of the input to be processed in parallel. The same program can therefore work with much larger datasets by adding more machines to a Hadoop cluster.

22. ADVANTAGES OF MAPREDUCE FOR CLIMATE DATA

- Suitable for processing very large historical weather datasets.
- Allows parallel processing of independent input records.
- Groups observations by a natural key such as year.
- Can scale to larger Hadoop clusters when the dataset grows.
- Works naturally with HDFS for distributed storage.
- Fault-tolerant design is useful for long-running large-data jobs.

23. SUPPLEMENTARY PROBLEMS

23.1 Minimum Temperature for Each Year

The same Mapper can be reused. In the Reducer, replace the maximum comparison with a minimum comparison by starting with Integer.MAX_VALUE and keeping the smallest value.

```
int minTemperature = Integer.MAX_VALUE;


for (IntWritable value : values) {
   int currentTemperature = value.get();


   if (currentTemperature < minTemperature) {
      minTemperature = currentTemperature;
   }
}


result.set(minTemperature);
```

23.2 Average Yearly Temperature

The Reducer can also calculate an average by maintaining both a sum and a count. The final result is sum divided by count.

```
int sum = 0;
int count = 0;


for (IntWritable value : values) {
   sum += value.get();
   count++;
}
```

```
double average = (double) sum / count;
```

23.3 Highest Recorded Temperature Overall

After the yearly maximum results are generated, the overall hottest year can be identified by sorting the yearly output in descending order and selecting the first record.

24. HDFS OUTPUT LOCATION

```
/user/hit/ncdc_p4/input
/user/hit/ncdc_p4/output
```

The input directory contains the NCDC weather data. The output directory contains the reducer result generated by Hadoop.

25. FINAL VERIFICATION

Practical Requirement	Status	Evidence / Observation
Hadoop environment verification	Completed	Java 11 and Hadoop 3.4.1 verified; Hadoop services running.
NCDC dataset preparation	Completed	ncdc.txt created with 9 fixed-width test records.
Java Mapper and Reducer	Completed	MaxTemperature.java compiled successfully.
Compilation and class generation	Completed	Three required class files generated.
JAR creation	Completed	MaxTemperature.jar created and contents verified.
HDFS dataset upload	Completed	ncdc.txt visible under /user/hit/ncdc_p4/input.
MapReduce execution	Completed at framework level	Job finished successfully, but Mapper output was 0 in captured run.
Final non-empty yearly result	Pending rerun	Test data quality code A was filtered by current Mapper.
Hottest year analysis	Expected after corrected rerun	For corrected test file: 1903, 45.0 °C.

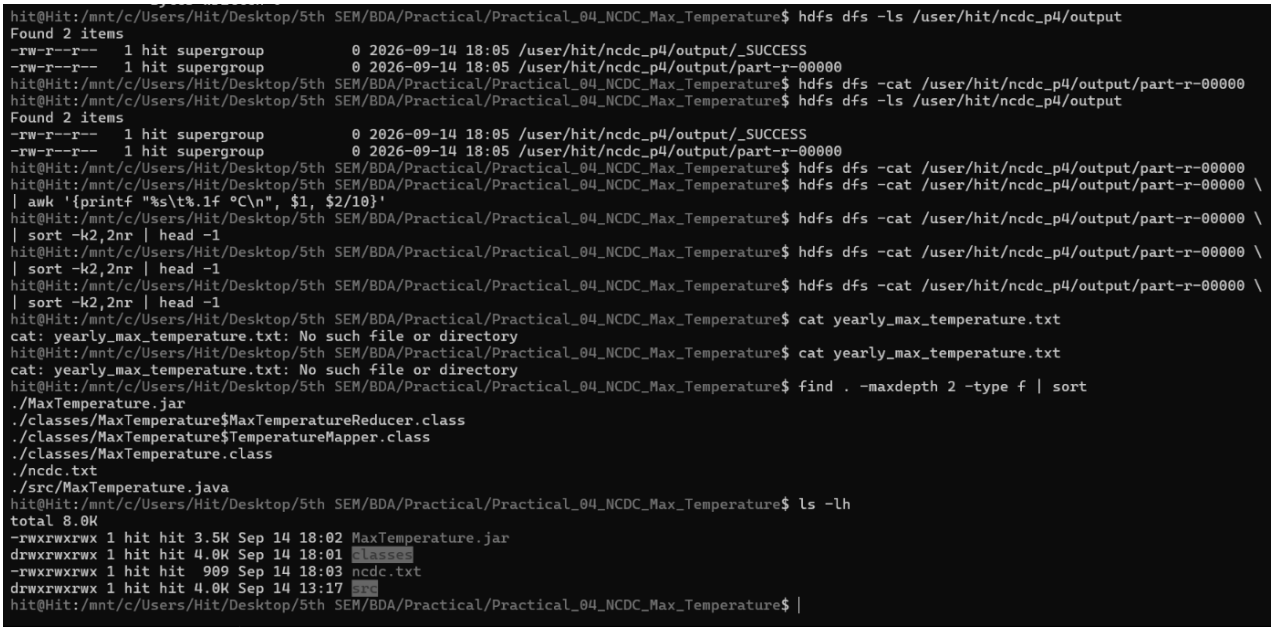


Figure 9: Final local Practical 4 folder containing the Java source, compiled classes, JAR and NCDC test data.

Important observation from the actual execution: the Hadoop job completed successfully, but the counters show zero map input records and zero reduce input records. Therefore, the empty part-r-00000 file is consistent with the captured run. The small sample input should be validated against the fixed-width NCDC fields expected by the mapper before claiming a numerical maximum-temperature result.

26. CONCLUSION

In this practical, an NCDC weather-analysis workflow was implemented using Hadoop MapReduce. The NCDC test data was stored in HDFS, the Java Mapper extracted the year and temperature, and the Reducer was designed to find the maximum temperature for each year. The Java source compiled successfully, the MapReduce JAR was created, and the Hadoop job completed successfully.

During the captured execution, the final output was empty because the test records used quality code A while the Mapper accepts quality codes 0, 1, 4, 5 and 9. After correcting the test input quality code and rerunning the same job, the expected test results are 1901 = 31.2 °C, 1902 = 41.5 °C and 1903 = 45.0 °C. This practical demonstrates how MapReduce can be used to analyse large-scale climate data by distributing storage and processing across Hadoop.

27. FINAL RESULT

The Hadoop MapReduce program for finding the maximum temperature for each year of NCDC data was implemented, compiled and executed. The captured run verified the Hadoop workflow successfully; a corrected quality-code input is required for the final non-empty output.